NAME: TANKISO MASOEBE

COMPUTER ARCHITECTURE(ADVANCED)WK10

1Personal Diary Program in C++

1. Introduction

The Personal Diary Program is a simple command-line application developed using C++. It allows users to write new diary entries and read previously saved entries. Each entry is saved in a text file together with the current date and time. The program uses file handling to ensure that diary entries remain saved even after the program is closed.

2. Objectives

- To allow users to add new diary entries.
- To automatically record the date and time of each entry.
- To save entries in a text file.
- To allow users to read all previously saved entries.

3. Source Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
using namespace std;

int main()
{
   int choice;
   string entry;

   do
   {
      cout << "\n===== PERSONAL DIARY =====" << endl;
      cout << "1. Add a new entry" << endl;
      cout << "2. Read previous entries" << endl;
      cout << "3. Exit" << endl;
      cout << "Enter your choice: ";
      cin >> choice;
      cin.ignore();

      if (choice == 1)
      {
         ofstream file("diary.txt", ios::app);
```

```
    if (file.is_open())
    {
        time_t now = time(0);
        char* dateTime = ctime(&now);

        cout << "\nEnter your diary entry: ";
        getline(cin, entry);

        file << "Date and Time: " << dateTime;
        file << "Entry: " << entry << endl;
        file << "--------------------------" << endl;

        file.close();

        cout << "Diary entry saved successfully!" << endl;
    }
    else
    {
        cout << "Error opening diary file." << endl;
    }
}
else if (choice == 2)
{
    ifstream file("diary.txt");

    if (file.is_open())
    {
        string line;

        cout << "\n===== PREVIOUS DIARY ENTRIES ====="
             << endl;

        while (getline(file, line))
        {
            cout << line << endl;
        }

        file.close();
    }
    else
    {
        cout << "No diary entries found yet." << endl;
    }
}
else if (choice == 3)
{
    cout << "Exiting the diary program." << endl;
}
else
{
    cout << "Invalid choice. Please try again." << endl;
}
```

```
    } while (choice != 3);

    return 0;
}
```

4. Explanation of the Program

a) Adding a new entry: The program asks the user to enter a diary entry. It uses ofstream to open the file named diary.txt and the ios::app mode to append new entries without deleting existing ones.

b) Recording the date and time: The time(0) and ctime() functions obtain the current date and time, which are saved together with the diary entry.

c) Reading previous entries: The program uses ifstream to open the diary file and getline() to read and display each line until the end of the file.

d) Menu system: A do-while loop displays the menu repeatedly until the user chooses option 3 to exit.

e) File handling: The program closes the file after reading or writing to it, helping ensure that file operations are completed properly.

5. Sample Output

```
===== PERSONAL DIARY =====
1. Add a new entry
2. Read previous entries
3. Exit
Enter your choice: 1

Enter your diary entry: Today I attended my programming class.
Diary entry saved successfully!

===== PERSONAL DIARY =====
1. Add a new entry
2. Read previous entries
3. Exit
Enter your choice: 2

===== PREVIOUS DIARY ENTRIES =====
Date and Time: Thu Oct 15 19:15:00 2026
Entry: Today I attended my programming class.
--------------------------

===== PERSONAL DIARY =====
1. Add a new entry
```

2. Read previous entries
3. Exit
Enter your choice: 3

Exiting the diary program.

6. Conclusion

The Personal Diary Program demonstrates the use of C++ file handling, loops, conditional statements, and date and time functions. It allows users to save diary entries, keep a record of when they were written, and read them later. The program is simple to use and meets the main requirements of the assignment.

2. Producer-Consumer Problem Using C++

1. Introduction

The Producer-Consumer Problem is a classic synchronization problem in operating systems. It involves two types of threads: producers and consumers. Producers create items and place them in a shared buffer, while consumers remove items from the buffer for processing.

The main challenge is ensuring that producers do not add items when the buffer is full and consumers do not remove items when the buffer is empty. Mutexes and semaphores are used to coordinate access to the shared buffer.

2. Objectives

- To implement producer and consumer threads in C++.
- To use a shared queue as a buffer.
- To use a mutex to protect the shared buffer.
- To use semaphores to control access when the buffer is full or empty.
- To demonstrate thread synchronization.

3. Source Code

The following program uses C++20, which supports counting semaphores.

```
#include <iostream>
#include <queue>
#include <thread>
```

```
#include <mutex>
#include <semaphore>
#include <chrono>

using namespace std;

const int BUFFER_SIZE = 5;
const int TOTAL_ITEMS = 10;

queue<int> buffer;

mutex bufferMutex;

counting_semaphore<BUFFER_SIZE> emptySlots(BUFFER_SIZE);
counting_semaphore<BUFFER_SIZE> fullSlots(0);

void producer()
{
   for (int i = 1; i <= TOTAL_ITEMS; i++)
   {
      emptySlots.acquire();

      bufferMutex.lock();

      buffer.push(i);

      cout << "Producer added item: "
           << i << endl;

      bufferMutex.unlock();

      fullSlots.release();

      this_thread::sleep_for(
         chrono::milliseconds(100));
   }
}

void consumer()
{
   for (int i = 1; i <= TOTAL_ITEMS; i++)
   {
      fullSlots.acquire();

      bufferMutex.lock();

      int item = buffer.front();
      buffer.pop();

      cout << "Consumer removed item: "
           << item << endl;

      bufferMutex.unlock();
```

```
        emptySlots.release();

        this_thread::sleep_for(
            chrono::milliseconds(150));
    }
}

int main()
{
    cout << "Producer-Consumer Problem" << endl;
    cout << "-------------------------" << endl;

    thread producerThread(producer);
    thread consumerThread(consumer);

    producerThread.join();
    consumerThread.join();

    cout << "All items have been processed."
         << endl;

    return 0;
}
```

4. Explanation of the Program

a) Shared buffer: The program uses a queue named buffer to store items produced by the producer and consumed by the consumer.

b) Producer thread: The producer creates items from 1 to 10 and adds them to the queue. Before adding an item, it checks whether an empty slot is available.

c) Consumer thread: The consumer removes items from the queue. It waits until an item is available before removing it.

d) Mutex: The bufferMutex protects the shared queue. It ensures that only one thread accesses or modifies the queue at a time.

e) Semaphores: The program uses two counting semaphores:

- emptySlots counts the available spaces in the buffer. It starts at 5 because the buffer is initially empty.
- fullSlots counts the items available for consumption. It starts at 0 because there are initially no items.

f) Thread synchronization: The acquire() function waits for a semaphore resource, while release() signals that a resource is available. The join() function makes the main program wait until both threads finish their work.

5. Sample Output

The output order may vary because the producer and consumer run concurrently.

```
Producer-Consumer Problem
-------------------------
Producer added item: 1
Consumer removed item: 1
Producer added item: 2
Consumer removed item: 2
Producer added item: 3
Consumer removed item: 3
Producer added item: 4
Consumer removed item: 4
...
Producer added item: 10
Consumer removed item: 10
All items have been processed.
```

6. Conclusion

The Producer-Consumer Problem demonstrates how synchronization is used when multiple threads share resources. In this program, the mutex prevents simultaneous modifications to the queue, while semaphores prevent the producer from adding items to a full buffer and the consumer from removing items from an empty buffer. Therefore, the program demonstrates the basic principles of thread synchronization in C++.

7. Compilation

This program requires a C++20-compatible compiler. If you are using GCC on a system with POSIX threads, compile it using:

```
g++ -std=c++20 producer_consumer.cpp -o producer_consumer -pthread
```

Then run it using:

```
./producer_consumer
```

3. Research: OpenMP vs. MPI in High-Performance Computing

1. Introduction

OpenMP and MPI are two important programming technologies used in High-Performance Computing (HPC). They allow programs to perform multiple tasks at the same time, improving processing speed and efficiency. However, they use different methods to achieve parallel processing.

2. OpenMP

OpenMP (Open Multi-Processing) is an API used for parallel programming on shared-memory systems. It allows multiple threads to work on different parts of a program while sharing the same memory space. OpenMP supports programming languages such as C, C++, and Fortran. It is relatively easy to use because programmers can add compiler directives to existing code to enable parallel execution.

One major advantage of OpenMP is that it is simple to implement and allows threads to share data easily. However, it is mainly suitable for computers with multiple processor cores that share memory. Shared data can also cause problems such as race conditions if synchronization is not handled correctly.

3. MPI

MPI stands for Message Passing Interface. It is a communication standard used in parallel and distributed computing. MPI allows multiple processes to communicate by sending and receiving messages. Each process generally has its own separate memory space, making MPI suitable for computer clusters and large supercomputers.

The main advantage of MPI is that it can scale across multiple computers, allowing it to solve very large problems. However, it can be more difficult to program because developers must manage communication between processes. Message passing can also introduce communication overhead, which may reduce performance.

4. Comparison and Applications

OpenMP is best suited for applications running on a single computer with multiple CPU cores and shared memory. Examples include image processing and scientific calculations. MPI is more suitable for applications that require several computers to

work together, such as weather forecasting, climate modelling, and large scientific simulations.

OpenMP is generally easier to learn and can reduce programming complexity, while MPI provides greater flexibility for distributed systems. OpenMP programs may face memory and scalability limitations on a single machine, whereas MPI programs must manage communication and data distribution carefully.

Both technologies can also be combined in hybrid programming, where MPI distributes work across computers and OpenMP creates threads within each computer.

5. Conclusion

In conclusion, OpenMP and MPI are both useful for parallel computing, but their suitability depends on the system and application. OpenMP is preferable for shared-memory systems because it is easier to implement, while MPI is preferable for distributed systems because it supports communication between computers and large-scale processing. The choice depends on performance requirements, system architecture, and programming complexity.